Mini actividad Clase 10 – Refactoring + Tests Unitarios

Proyecto: Mi Backlog (TP1) – biblioteca personal de juegos, series y películas.
Funcionalidad elegida: búsqueda, filtros y orden de la lista (función `renderLista` en `js/ui.js`).

---

1. Análisis del código

Prompt usado:

```
Leé la SPEC y el AGENTS.md del proyecto.
Analizá únicamente la funcionalidad de búsqueda, filtros y orden de la lista
(función renderLista en js/ui.js).
No escribas código todavía.
Detectá oportunidades de refactoring y proponé una mejora concreta.
```

Lo que detectó el análisis:

Problema	Dónde	Detalle
Duplicación	`renderLista`	La cadena `.toLowerCase().normalize('NFD').replace(...)` para sacar tildes está repetida **4 veces**.
Función demasiado larga	`renderLista`	51 líneas que hacen tres cosas: filtrar, ordenar y dibujar.
Responsabilidades mezcladas	`renderLista`	La lógica (qué títulos se ven y en qué orden) está mezclada con el DOM. Por eso **no se puede testear** sin abrir la página.
Nombres poco claros	`renderLista`	Variables como `t` y `resultado` no dicen qué contienen.
Oportunidad de simplificación	`renderLista`	Seis `if` encadenados que reasignan `resultado` se pueden reemplazar por un solo `filter` con una función que diga si un título cumple los filtros. El `if / else if / else` del orden se puede reemplazar por un objeto de comparadores.

2. Mejora elegida

Separar la lógica de filtrar y ordenar del renderizado (refactoring "Extraer función / módulo").

- Se creó `js/filtros.js` con funciones **puras** (reciben datos, devuelven datos, no tocan el DOM):
  - `normalizarTexto(texto)`: minúsculas y sin tildes, escrita una sola vez.
  - `filtrarYOrdenar(titulos, filtros)`: devuelve los títulos visibles en el orden pedido.
- `renderLista` en `ui.js` ahora solo **dibuja**: pide la lista ya filtrada y crea las tarjetas.

Por qué esta mejora: es la que resuelve el problema de raíz (responsabilidades mezcladas). De paso elimina la duplicación y acorta la función, y sobre todo hace que la lógica **se pueda testear sin navegador ni DOM**, que es lo que pide el paso 3.

Comportamiento: no cambió. Mismos filtros, mismas reglas de comparación y mismo orden.

Código antes del refactoring

`js/ui.js` – `renderLista` (versión original completa en `antes/ui.js`):

```
function renderLista(titulos, filtros) {
  const lista = $('lista');
  lista.replaceChildren();

  let resultado = titulos.slice();

  if (filtros.texto) {
    const busqueda = filtros.texto.toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g, '');
    resultado = resultado.filter((t) =>
      t.titulo.toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g, '').includes(busqueda)
    );
  }
  if (filtros.tipo) {
    resultado = resultado.filter((t) => t.tipo === filtros.tipo);
  }
  if (filtros.estado) {
    resultado = resultado.filter((t) => t.estado === filtros.estado);
  }
  if (filtros.plataforma) {
    const plataforma = filtros.plataforma.toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g, '');
    resultado = resultado.filter((t) =>
      (t.plataforma || '').toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g,
'').includes(plataforma)
    );
  }
  if (filtros.genero) {
    resultado = resultado.filter((t) => t.genero === filtros.genero);
  }
  if (filtros.soloFavoritos) {
    resultado = resultado.filter((t) => t.favorito);
  }

  if (filtros.orden === 'nombre') {
    resultado.sort((a, b) => a.titulo.localeCompare(b.titulo, 'es', { sensitivity: 'base' }));
  } else if (filtros.orden === 'puntaje') {
    resultado.sort((a, b) => (b.puntaje || 0) - (a.puntaje || 0));
  } else {
    resultado.sort((a, b) => b.fechaCreacion.localeCompare(a.fechaCreacion));
  }

  if (resultado.length === 0) {
    const vacio = document.createElement('li');
    vacio.className = 'vacio';
    vacio.textContent = titulos.length === 0
      ? 'Tu biblioteca está vacía. Agregá tu primer título con el formulario.'
      : 'Ningún título coincide con la búsqueda o los filtros.';
    lista.append(vacio);
    return;
  }

  resultado.forEach((titulo) => lista.append(crearTarjeta(titulo)));
}
```

Código después del refactoring

Nuevo `js/filtros.js`:

```
// Pasa a minúsculas y saca las tildes, para comparar "Película" con "pelicula".
function normalizarTexto(texto) {
  return String(texto ?? '').toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g, '');
}

function contieneTexto(textoCompleto, busqueda) {
  return normalizarTexto(textoCompleto).includes(normalizarTexto(busqueda));
}

// Un filtro vacío ('' o false) no se aplica.
function cumpleFiltros(titulo, filtros) {
  return (!filtros.texto || contieneTexto(titulo.titulo, filtros.texto))
    && (!filtros.tipo || titulo.tipo === filtros.tipo)
    && (!filtros.estado || titulo.estado === filtros.estado)
    && (!filtros.plataforma || contieneTexto(titulo.plataforma, filtros.plataforma))
    && (!filtros.genero || titulo.genero === filtros.genero)
    && (!filtros.soloFavoritos || Boolean(titulo.favorito));
}

const COMPARADORES = {
  nombre: (a, b) => a.titulo.localeCompare(b.titulo, 'es', { sensitivity: 'base' }),
  puntaje: (a, b) => (b.puntaje || 0) - (a.puntaje || 0),
  fecha: (a, b) => b.fechaCreacion.localeCompare(a.fechaCreacion)
};

// Devuelve un array nuevo; nunca modifica el original.
function filtrarYOrdenar(titulos, filtros) {
  const comparar = COMPARADORES[filtros.orden] || COMPARADORES.fecha;
  return titulos.filter((titulo) => cumpleFiltros(titulo, filtros)).sort(comparar);
}
```

`js/ui.js` – `renderLista` ahora solo dibuja:

```
function renderLista(titulos, filtros) {
  const lista = $('lista');
  lista.replaceChildren();

  const visibles = app.filtros.filtrarYOrdenar(titulos, filtros);

  if (visibles.length === 0) {
    const vacio = document.createElement('li');
    vacio.className = 'vacio';
    vacio.textContent = titulos.length === 0
      ? 'Tu biblioteca está vacía. Agregá tu primer título con el formulario.'
      : 'Ningún título coincide con la búsqueda o los filtros.';
    lista.append(vacio);
    return;
  }

  visibles.forEach((titulo) => lista.append(crearTarjeta(titulo)));
}
```

Resultado: `renderLista` pasó de 51 a 18 líneas, la normalización de texto está escrita una sola vez y la lógica quedó en un archivo aparte que se puede testear.

3 y 4. Tests unitarios

Archivo: `mi-backlog/tests/filtros.test.js` (18 tests). Se ejecutan abriendo `mi-backlog/tests/tests.html` en el navegador, con un mini runner propio (`tests/runner.js`) porque el proyecto no usa librerías.

Casos correctos

Test	Qué valida
Sin filtros devuelve todo, del más reciente al más antiguo	Que no se pierda ningún título y que el orden por defecto sea por fecha de creación descendente.
Busca por parte del nombre sin importar mayúsculas	"ZELDA" encuentra "The Legend of Zelda" (búsqueda parcial y sin distinguir mayúsculas).
Busca sin importar las tildes	"pelicula" encuentra "Película de prueba".
Combina varios filtros a la vez	Tipo + estado + favoritos se aplican juntos (condición Y).
Filtra por plataforma con coincidencia parcial	"net" encuentra los títulos de Netflix.
Ordena por nombre A-Z sin distinguir mayúsculas	"alien" (minúscula) queda primero, no al final.
Ordena por puntaje de mayor a menor	Los títulos sin puntaje quedan al final.
`normalizarTexto` pasa a minúsculas y saca tildes	"Acción Ñandú ÉPICO" → "accion nandu epico".

Casos con datos inválidos

Test	Qué valida
Un tipo que no existe ("Libro") no devuelve nada	Un valor de filtro inválido no rompe la app y no muestra resultados falsos.
Títulos sin plataforma ni favorito no rompen el filtro	Registros incompletos (campos `undefined`) no tiran error.
Un criterio de orden desconocido usa el orden por fecha	Si llega un `orden` inválido, se usa el orden por defecto.
`normalizarTexto` acepta `null` o `undefined`	Devuelve `''` en vez de lanzar un error.

Casos límite

Test	Qué valida
Biblioteca vacía devuelve una lista vacía	Funciona con cero títulos, aunque haya filtros activos.
Búsqueda sin coincidencias devuelve lista vacía	Caso "no hay resultados".
Un solo título se devuelve igual con cualquier orden	Los tres criterios de orden funcionan con un solo elemento.
Puntajes empatados mantienen su orden original	El orden es estable: no "mezcla" los títulos que empatan.
No modifica el array original	`filtrarYOrdenar` no altera los datos guardados (importante porque `sort` modifica el array).
Puntajes en los extremos (1 y 10)	Los valores mínimo y máximo y el "sin puntaje" quedan en el orden correcto.

5. Resultado de la ejecución

Ejecutados después del refactoring:

```
18/18 tests pasaron
✓ [correcto] Sin filtros devuelve todo, del más reciente al más antiguo
✓ [correcto] Busca por parte del nombre sin importar mayúsculas
✓ [correcto] Busca sin importar las tildes ("pelicula" encuentra "Película")
✓ [correcto] Combina varios filtros a la vez (tipo + estado + favoritos)
✓ [correcto] Filtra por plataforma con coincidencia parcial ("net" → Netflix)
✓ [correcto] Ordena por nombre A-Z sin distinguir mayúsculas
✓ [correcto] Ordena por puntaje de mayor a menor; los que no tienen puntaje van al final
✓ [correcto] normalizarTexto pasa a minúsculas y saca tildes
✓ [inválido] Un tipo que no existe ("Libro") no devuelve nada
✓ [inválido] Títulos sin plataforma ni favorito no rompen el filtro de plataforma
✓ [inválido] Un criterio de orden desconocido usa el orden por fecha
✓ [inválido] normalizarTexto acepta null o undefined y devuelve texto vacío
✓ [límite] Biblioteca vacía devuelve una lista vacía
✓ [límite] Búsqueda que no coincide con nada devuelve lista vacía
✓ [límite] Un solo título se devuelve igual con cualquier orden
✓ [límite] Puntajes empatados mantienen su orden original (orden estable)
✓ [límite] No modifica el array original al filtrar y ordenar
✓ [límite] Puntajes en los extremos (1 y 10) se ordenan bien
```

Verificación de que el comportamiento no cambió

Antes de los tests unitarios (que solo existen para el código nuevo), se comparó la app **antes y después** del refactoring en el navegador. Con los mismos 5 títulos de ejemplo se aplicaron **14 combinaciones** de búsqueda, filtros y orden (sin filtros, cada orden, búsqueda con y sin tildes, sin resultados, cada filtro solo y varios combinados) y se anotó qué títulos mostraba la lista y en qué orden.
Las 14 combinaciones dieron exactamente el mismo resultado antes y después.

Verificación de que los tests detectan errores

Para comprobar que los tests no pasan "porque sí", se rompió a propósito una copia de `filtros.js` (se invirtió el orden por puntaje). Resultado:

```
16/18 tests pasaron
✗ [correcto] Ordena por puntaje de mayor a menor; los que no tienen puntaje van al final
✗ [límite] Puntajes en los extremos (1 y 10) se ordenan bien
```

Los tests detectaron el error. Después se descartó la copia rota.

Cómo correr los tests

Abrir `mi-backlog/tests/tests.html` con doble clic. La página muestra cada test con ✓ o ✗, y la pestaña dice `PASS 18/18` o `FAIL n/18`.